

```
@page { margin: 14mm 12mm; } body { font-family: "DejaVu Sans",  
Arial, sans-serif; font-size: 10.5pt; line-height: 1.34; color: #1a2733;  
max-width: 100%; margin: 0; } h1 { font-size: 19pt; color: #12324b;  
border-bottom: 2px solid #12324b; padding-bottom: 3px; } h2 { font-  
size: 14pt; color: #12324b; border-left: 4px solid #12324b; padding-  
left: 7px; margin-top: 16px; } h3 { font-size: 12pt; color: #234;  
margin-top: 12px; } table { border-collapse: collapse; width: 100%;  
margin: 8px 0; font-size: 9pt; } th, td { border: 1px solid #b7c2cc;  
padding: 3px 6px; text-align: left; vertical-align: top; } th {  
background: #12324b; color: #fff; } tr:nth-child(even) td {  
background: #f2f5f7; } code { background: #eef1f4; padding: 1px  
3px; border-radius: 3px; font-size: 8.8pt; } pre { background:  
#0f1b26; color: #dfe7ee; padding: 8px 10px; border-radius: 5px; font-  
size: 8.4pt; overflow-x: auto; white-space: pre-wrap; } pre code {  
background: transparent; color: inherit; }
```

VERIFICATION_REFERENCE

O-RAN VIP Suite — Verification Reference Document

Author: AVIK MAJUMDAR **Project:** AVIK VIP FACTORY — O-RAN VIP Suite **Document:** Verification Reference (VRD) **Revision:** 1.0 **Date:** 2026-08-09

1. Purpose and Scope

This Verification Reference Document (VRD) defines the verification methodology, environment architecture, gate model, stimulus and coverage strategy, assertion strategy, traceability model, and signoff criteria for the O-RAN VIP Suite. The suite verifies thirteen O-RAN fronthaul components spanning 4G LTE (CPRI-over-Ethernet) and 5G NR (eCPRI Split 7.2x) as a single unified product with shared infrastructure.

The document is the authoritative reference for engineers extending, running, or signing off the suite. It is written to be tool-portable: the same methodology runs on Verilator 5.020 (free), Verilator 5.050 (mandated primary, source-built), and any UVM-capable commercial simulator (VCS, Questa, Xcelium).

1.1 Components in scope

#	Component	Mode	Function
1	ecpri_transport	Both	eCPRI transport layer — common header, message types 0-7, sequence numbering
2	cpri_eth	4G LTE	CPRI-over-Ethernet, 6 bandwidth profiles, SyncE, C&M slow plane
3	uplane	5G NR	U-Plane IQ delivery, Section Types 1/3/5/6, PRB map, compression
4	cplane	5G NR	C-Plane scheduling, Section Types 0-8, Section Extensions 1-

			11
5	splane	Both	S-Plane IEEE 1588v2 PTP, SyncE, timing error budget
6	mplane	Both	M-Plane NETCONF/YANG, O1 interface, RU config
7	beamforming	5G NR	Beamforming weights, codebook, Section Extensions 1/4/5/6
8	compression	5G NR	IQ compression — BFP, mu-law, static, bit-width reduction
9	prach	5G NR	PRACH handler, Section Type 3, ZC sequence, formats 0/A/B/C
10	mimo_massive	5G NR	Massive MIMO up to 64T64R, layer mapping, TDD config
11	bwp	5G NR	BWP manager, dynamic switch, numerology per BWP, FR1/FR2
12	mmwave	5G NR	mmWave FR2, bands n257/n258/n260/n261, mu=2/3/4
13	laa	5G NR	LAA handler, Section Type 5, listen-before-talk

1.2 Applicable specifications

eCPRI Specification v2.0; O-RAN.WG4.CUS.0 (C/U/S-plane); O-RAN.WG4.MP.0 (M-plane); IEEE 1914.3 (Radio over Ethernet); IEEE 1588v2 (PTP); 3GPP TS 38.211 (NR physical layer); 3GPP TS 36.211 (LTE physical layer). Requirements are currently **DERIVED** from these sources; they reconcile to authoritative O-RAN.WG4.CUS.0 R004-v16.01 clause citations when the revision is pinned (GAP-ORAN-001).

2. Verification Environment Architecture

The suite uses a three-lane architecture. Each lane earns its evidence on the simulator capable of executing it; together they cover the full VIP stack. There is no fourth lane — “Lane-2” is the UVM tier, split into A (subset) and B (native).

2.1 Lane-1 — plain-SystemVerilog VIP (Verilator)

Purpose: verify protocol field encoding, packing/unpacking, legal-space enforcement (per-field and cross-field), negative/illegal detection, and functional + code coverage.

- **Simulator:** Verilator 5.020 (apt) or 5.050. Verilator 5.020 does **not** honor SV `randomize()` constraints (measured: %Warning-CONSTRAINTIGN, values fall outside the declared legal space). Therefore constrained-random is driven externally.
- **Stimulus:** Z3 constraint solver pre-generates seeded, legal-by-construction stimulus. The Z3 model **is** the constraint block. Reproducible by seed (identical md5 for identical seed; divergence for different seeds).
- **DUT-equivalent:** a combinational codec — `pack(fields) → 64-bit wire` and `unpack(wire) → fields` — plus `is_legal()` and

- first_violation().
- **Scoreboard:** three independent checks — runtime legality self-check (counters the silent-CONSTRAINTIGN hazard), pack-versus-golden cross-check (two independent packers must agree), and round-trip identity.
- **Coverage:** functional via a monitor-log COV-### engine; code (line/branch/toggle) via a custom sim_main.cpp that calls coveragep()->write("coverage.dat").
- **Reporter:** UVM-format summary; STATUS adjudicated from executed evidence only.

2.2 Lane-2A — UVM component architecture on Verilator (muvm_pkg)

Purpose: exercise the UVM **structural** layer that plain-SV cannot — factory (with override), config_db, build/connect/run/report phasing, analysis-port TLM, and a uvm_component-style scoreboard. muvm_pkg is a UVM-1.1-compatible subset hand-built to elaborate and run on Verilator's class engine (full Accellera UVM does not run on 5.020). It reuses the Lane-1 pack/unpack DUT and Z3 stimulus, driving transactions through a real sequencer/driver to a component scoreboard.

2.3 Lane-2B — native Accellera UVM (Verilator 5.050 / commercial)

Purpose: the full native UVM tier — native randomize() with {} constraint solving, native covergroup with cross and illegal_bins, and concurrent SVA temporal properties. Verilator 5.050 (source-built) runs full Accellera UVM via a small DPI shim (Section 8.3). The identical source runs on VCS, Questa, and Xcelium.

2.3.1 RAL — register abstraction layer (uvm_reg, native UVM)

Purpose: model-based register verification of the O-RAN M-plane RU configuration space using the standard uvm_reg model. Built as native Accellera UVM and run on the source-built Verilator 5.050 (identical source runs on VCS/Questa/Xcelium). The register model (lane2/ral/oran_ral_pkg.sv) declares five registers with typed RW/RO fields, an uvm_reg_block with a little-endian reg_map, an uvm_reg_adapter (reg2bus/bus2reg), a memory-modelling driver, and a front-door sequence exercising randomize→update→read→mirror→compare with set_auto_predict. Register map:

Register	Offset	Access	Fields (bit-range)
ru_ctrl_reg	0x00	RW	enable[0], mode[2:1], antmap[10:3]
ru_version_reg	0x04	RO	major[7:0]=0x02, minor[15:8]=0x0A
ru_numant_reg	0x08	RW	n_tx[6:0], n_rx[14:8]
ru_comp_reg	0x0C	RW	method[1:0], width[6:2]=16
ru_bwp_reg	0x10	RW	bwp_id[1:0], numerology[5:2], active[6]

Checks: reset-value mirror, RO version read-back, 20× per-register RW randomize/update/ read/mirror-compare (front-door via adapter), and RO write-protect (write to ru_version must not change it). Executed evidence: oran_ral_test PASS, 163 register operations, 0 errors, 0 UVM_ERROR, 0 UVM_FATAL on Verilator 5.050. Run: make ral SIM=verilator5050.

2.4 Shared infrastructure

A single component descriptor (spec.json, generated from gen/build_suite.py SPECS) is the source of truth for all three lanes: field widths, legal space, cross-field constraints, and coverage bins. Lane-1 TB, Lane-2A UVM-subset env, and Lane-2B native UVM env are all generated from it, guaranteeing consistency across lanes.

3. Gate Model (per component)

No gate is skipped. GATE 0 and GATE 1 artifacts must exist before GATE 2 can be claimed; equally, gate N cannot be entered until N-1 is met. Every STATUS is derived from executed simulator evidence.

Gate	Name	Entry criteria	Exit criteria / evidence
0	Requirements	spec available (DERIVED acceptable)	vplan with FR-### → COV/SVA mapping; no in-scope clause without an FR
1	Architecture	GATE 0 complete	env hierarchy, scoreboard strategy, coverage model, SVA list, timing budget
2	Compile (L0)	GATE 1 complete	clean compile log (never from inspection)
3	Smoke (L1)	GATE 2 PASS	100% PASS on seeds 1,2,3 in result.json
4	Directed (L2)	GATE 3 PASS	zero hardcoded literals in drive path (scan); one directed scenario per FR
5	Random (L3)	GATE 4 PASS	100 seeds, ≥99% PASS, coverage convergence recorded
6	Negative	GATE 5 PASS	EXPECTED_FAILURE_DETECTED for each illegal condition; fired check named
7	Assertions	GATE 6 PASS	every invariant exercised ≥1 (non-vacuous); zero disabled unless waived
8	Coverage	GATE 7 PASS	functional ≥95% merged; code stmt ≥90, branch ≥85 (waivers documented)
9	Full regression (L5)	GATE 8 PASS	≥99% PASS across all tests/seeds; regression DB updated
10	Review	GATE 9 PASS	signoff section complete: gate results, coverage, portability, waivers
11	Signoff	GATE 10 complete	Verilator=PASS; other simulators PASS/NOT_RUN, explicit

3.1 Regression tiers

L0 compile · L1 smoke (seeds 1,2,3) · L2 directed (one per FR) · L3 random (100 seeds) · L4 negative · L5 full (all classes, all seeds).
Makefile targets: regress-l1..l5, regress (all lanes).

4. Stimulus and Constrained-Random Strategy

4.1 Constrained-random discipline (non-negotiable)

No hardcoded field values in any sequence or test. All values are produced by a constraint solver: Z3 (Lane-1/2A) or native `randomize()` (Lane-2B). Constraints define the legal space; corner cases are forced by tightening constraints (directed mode), never by literals. `uvm_fatal/$fatal` on any failed `randomize`. Seed controls the run — the same seed reproduces exactly.

4.2 The Z3 mechanism (Lane-1/2A)

Because Verilator 5.020 silently ignores SV constraints, the Z3 model is authoritative. Each field is encoded as a BitVec with per-field legality (`const/enum/range`) and cross-field constraints as Z3 relations. The solver emits N seeded, legal transactions plus an independent golden packed word. A runtime legality self-check in the TB verifies every transaction against the legal predicate, so any escape is caught rather than silently passing.

4.3 Native randomize (Lane-2B)

On Verilator 5.050 and commercial simulators, `randomize()` with `{}` is honored (measured on 5.050: 0/200 violations, z3-backed solver). The same legal space and cross-constraints are expressed as native SV constraint blocks. The scoreboard independently re-checks legality (defensive), and native covergroup `illegal_bins` must never be hit under legal stimulus.

4.4 Cross-field constraints (the solver showcase)

Component	Cross-constraint	Verified
uplane	$\text{start_prb} + \text{num_prb} \leq 275$	0 violations / 900+ txns
cplane	$\text{start_symbol} + \text{num_symbol} \leq 14$	0 violations
beamforming	$\text{num_layers} \leq \text{num_ports}$	0 violations
compression	$\text{comp_width} \leq \text{iq_width}$	0 violations
mimo_massive	$\text{num_layers} \leq \text{ant_cfg}; \text{rank} \leq \text{num_layers}$	0 violations

These relations cannot be enforced by a simulator that ignores constraints; they are the primary demonstration of the Z3 backend on Lane-1 and native solving on Lane-2B.

5. Coverage Methodology

5.1 Functional coverage

Every covered protocol field maps to a COV-### coverpoint: `enum` → one bin per legal value; `bool` → {0,1}; `wrap` → {min, max, mid}; `bucket4` → four range buckets. Cross coverage is defined on the two most significant coverpoints. Coverage is merged across smoke, random (100 seeds), directed, and corner classes. Target ≥95% merged; achieved **100%** for all thirteen components. Wide fields unreachable by random (e.g. a 16-bit sequence field's exact max) are closed by a directed seed constraining that field.

5.2 Native covergroup (Lane-2B)

Lane-2B uses native SV covergroup with coverpoints, `cross`, and `illegal_bins`. The `illegal_bins` (values outside the legal set) must remain empty under legal stimulus — their emptiness is positive evidence that the native constraints held.

5.3 Code coverage

Line, branch, and toggle coverage are collected via Verilator's --coverage and a custom sim_main.cpp that flushes coverage.dat at end of simulation (the built-in --binary main omits this flush — resolved as GAP-ORAN-003). Positive, negative, and a corrupt-golden checker self-test are merged. Achieved: statement (line) **100%**; DUT branch **100%**. Six per-component branches in the testbench harness are defensive error handlers (\$fatal no-STIM / no-file, mismatch handlers, fail-summary) that execute only on an infrastructure fault; these are formally **waived** with rationale — zero DUT-logic branches remain uncovered. Toggle coverage is reported for completeness (not a GATE-8 criterion; low on wide data fields under bounded random, as expected).

6. Assertion Methodology

6.1 Lane-1 / Lane-2A — procedural invariant checkers

Each invariant (recovered version == 0x1, msg_type ≤ 7, payload bounds, round-trip identity, pack-versus-golden) is a procedural check with an exercise counter. A check with zero firings across regression is vacuous and fails GATE 7. Every invariant family exercises ≥300 times per seed (non-vacuous).

6.2 Lane-2B — concurrent SVA

Temporal properties (version invariant, msg_type bound, payload bound, sequence-number continuity) are written as concurrent assert property in a bound interface, each paired with a cover property for the vacuity check. Concurrent SVA is supported on Verilator 5.050 and all commercial simulators.

6.3 Negative testing (GATE 6)

Negative mode injects a guaranteed-illegal value (out-of-range, out-of-enum, wrong constant, or cross-constraint violation) into each transaction. The checker must detect it and the run reports EXPECTED_FAILURE_DETECTED with the fired check named. A checker that stays silent on illegal stimulus is an UNEXPECTED_PASS and fails.

7. Traceability and Result Policy

7.1 Traceability chain

FR-### (requirement) → FEAT-### (feature) → VC-### (verification component) → SEQ-### (sequence) → COV-### / SVA-### (coverage / assertion) → BUG-### (defect). The per-component GATE 0 vplan carries the FR → COV/SVA table.

7.2 PASS / FAIL policy (non-negotiable)

Every run emits result.json via adjudicate.py. STATUS = PASS iff simulator exit == 0 AND UVM_ERROR == 0 AND UVM_FATAL == 0 AND transactions > 0. Negative test STATUS = EXPECTED_FAILURE_DETECTED. A missing or unparseable log = NOT_VERIFIED, which is never converted to PASS. Compile ≠ PASS; a claimed result ≠ PASS; only executed, observable evidence counts.

8. Simulator Portability

8.1 Executed matrix

Simulator	Lane-1	Lane-2A	Lane-2B
Verilator 5.020	PASS 1470/1470	PASS 39/39	N/A (no UVM)
Verilator 5.050	PASS	PASS	PASS 13/13
VCS / Questa / Xcelium	NOT_RUN	NOT_RUN	source-ready (same UVM)

NOT_RUN is not PASS and is stated explicitly.

8.2 Verilator 5.050 measured capability

Native randomize() with {} HONORED (0/200 violations); covergroup + cross + illegal_bins SUPPORTED; concurrent SVA WORKS; full Accellera UVM ELABORATES and RUNS.

8.3 UVM-on-Verilator DPI shim

Full UVM links on Verilator 5.050 with three adaptations (persisted in lane2/uvm050/): no-op HDL-backdoor stubs whose signatures match uvm_dpi.h (char*, p_vpi_vecval); the VPI polling include disabled (register backdoor unused); and a vpi_get_vlog_info stub for the tool-name query. With these, run_test executes real UVM phasing, factory, and the report server.

9. Executed Results (2026-08-09)

Lane-1 1470/1470 · Lane-2A 39/39 · Lane-2B 13/13 (native UVM, Verilator 5.050) · RAL 1/1 (oran_ral_test, 163 reg ops, 0 err).

Combined: 1523/1523 = 100% real executed pass. Functional coverage 100% merged; code statement 100%, DUT branch 100% (TB-defensive branches waived). Detailed per-component results are in the Full Regression Report and regression/combined_regression.json.

10. Signoff Criteria and Open Items

GATE 11 signoff requires all gates 0-10 met with Verilator=PASS and other simulators PASS/NOT_RUN stated explicitly. Open, non-blocking items:

- **GAP-ORAN-001** (requirements): GATE-0 requirements are DERIVED; they reconcile to authoritative O-RAN.WG4.CUS.0 R004-v16.01 clause references when the revision is pinned.
- **GAP-ORAN-002** (portability): commercial-simulator execution is optional; the identical native-UVM source runs on VCS/Questa/Xcelium via make lane2b SIM=<sim>.
- **GAP-ORAN-003** (code-coverage flush) and **GAP-ORAN-004** (functional holes): RESOLVED.